Feasibility Plan and Improvement Suggestions: Adaptive Energy-Aware PC/BP Selection

Oct 9, 2026 · @Adi

Summary

Your MNIST phase shows the controller logic works, but it does not yet prove the energy claim your professor needs to see. The Azure and COCO run can validate accuracy behavior; only real edge hardware can validate energy.

- **What MNIST shows:** with fixed seeds, the hybrid keeps ~97.5% accuracy under disturbance while Always-PC drops to ~12%, at ~48.5% less energy than Always-BP (1.80 J vs 3.50 J over 29 steps).
- **What it does not show:** that PC is cheaper than BP on real edge hardware, that the saving holds outside a synthetic 29-step run (it depends on how many steps you put in BP), or that the free-energy trigger works on YOLO features.
- **The plan below:** fix the dataset and hardware roles, close six conceptual gaps, add stronger baselines, and agree three pass/fail gates with your professor before running anything.

Azure, edge hardware and datasets

Use Azure for accuracy and the cloud-BP tier, and use a Raspberry Pi 5 for every energy number. A cloud VM has no PMIC and usually no RAPL access, so any Joules measured there are estimates.

- **Azure can show:** mAP behavior, the free-energy signal, and cloud BP retraining. On a GPU VM, integrate NVML power draw over time. On a CPU VM, CodeCarbon only gives TDP-based estimates.
- **The Pi 5 must show:** J/frame for inference, one PC step and one BP step. A Pi 5 plus a USB-C power meter costs under 100 euros, and it is where the energy claim is proven or killed. Run this microbenchmark first, not in weeks 5-6.
- **Jetson Nano:** it is stuck on old JetPack and Python versions that struggle with current Ultralytics. Use an Orin Nano if you can get one, otherwise drop it.

- **Azure cost control:** use a spot or T4 VM, deallocate it when idle, and set a budget alert.

COCO is a set of still images, so it has no video and no distribution shift. YOLOv8n was pre-trained on COCO, so clean COCO val is in-distribution and neither learning rule will show a gain.

- **Controlled shifts:** build COCO val into streams with scheduled corruptions (blur, brightness drop, occlusion, fog) using the `imagecorruptions` package. A frozen YOLOv8n will lose mAP there, which gives you something to recover.
- **A natural shift:** add one real video set with day/night or weather changes, such as BDD100K, or a fixed-camera set such as UA-DETRAC.
- **A sanity check:** clean-COCO mAP should match the Ultralytics published figure for YOLOv8n (about 37 mAP50-95; confirm in their docs).

Six gaps to close before more experiments

These are the points a committee is most likely to press.

1. **Is PC really cheaper on this hardware?** Ten relaxation steps per sample is usually more arithmetic than one BP step. Your MNIST gap (1.5 ms vs 50 ms) probably comes from implementation differences or simulated cloud latency, so check it. If PC is not cheaper on a Pi CPU, your RQ2 break-even analysis becomes the finding, and that is still a valid thesis.
2. **Where do the labels come from?** Cloud BP on surveillance video needs labels, and none exist in deployment. State the source, for example pseudo-labels from a larger YOLO teacher. PC minimizes feature reconstruction error and does not directly improve mAP, so say what it adapts.
3. **The switching rule ignores J.** Your policy thresholds only F, so α, β, γ, δ never affect a decision, and H is redundant with τ (only τ_upper + H and τ_lower - H matter). The "6D space" is really about 2 effective thresholds. Either make the rule truly J-based (estimate each mode's cost and switch when the gap exceeds a margin) or state that the weights only shape the Pareto analysis. Also normalize each term of J, since it mixes Joules, ms, accuracy and F.
4. **Fixed thresholds will not transfer.** τ = 0.70/0.30 was tuned on MNIST. For YOLO features, normalize F against a running baseline (EWMA or z-score), and show that F tracks mAP drop (for example by AUROC) before relying on it.

5. **Anomalies versus disturbances.** An intruder entering the scene also spikes F, and a PC layer that adapts to it will learn to predict the intruder away. Separate global errors (lighting, blur) from localized ones (objects), and do not adapt on localized spikes.
6. **Energy accounting.** Decide whether E is edge-only or system-wide. Mode 1 costs the edge radio transmission and the weight download, and both need modeling. Also define what happens when the network is down.

Baselines and statistics

The hybrid has to beat a duty-cycle-matched baseline, otherwise the controller's intelligence is not shown.

- **Frozen model:** the floor for mAP under corruption.
- **BN-statistics adaptation or TENT:** a cheap test-time adaptation method. A committee will ask why you did not use it.
- **On-device BP on the head only:** the cheap form of BP, with low memory.
- **Always-PC with a fair tuning budget:** Always-PC at ~12% looks like a strawman as it stands.
- **Always-cloud-BP:** the accuracy upper bound.
- **Periodic or random switcher with the same BP duty cycle as the hybrid:** shows whether the free-energy trigger adds anything.

On statistics:

- Your 48.5% saving reflects roughly 40% of steps in BP, a share you chose. Report saving as a curve over disturbance fraction.
- Use at least 5 seeds and report 95% confidence intervals. Replace "100% deterministic consistency" with "fixed seeds, identical curves".
- On the Pi, subtract idle power, warm up first, and log `vcgencmd get_throttled`.

Feasibility plan for the professor

Feasibility will be judged by three pass/fail gates, agreed with the supervisor before any experiment runs. Gates 2 and 3 can turn a negative result into a quantified break-even point, which still answers RQ2.

Gate	Where	What is run	Pass criterion	If it fails
1. Signal	Azure, corrupted COCO streams	Frozen YOLOv8n vs hybrid on scheduled corruptions; log F and mAP per segment	F detects mAP degradation (AUROC of about 0.9 or higher), and the hybrid recovers most of the mAP gap between frozen and always-BP	Change the trigger (normalized F, spatial F map) before moving to hardware
2. Energy	Raspberry Pi 5 with a USB-C power meter	J/frame for inference only, one PC step and one BP step, idle power subtracted, 5 or more repeats	PC is cheaper than BP at matched adaptation quality, or the break-even point is quantified	Report the break-even conditions and reframe the contribution around them
3. End to end	Pi 5 plus Azure, network emulated with `tc netem`	Hybrid vs periodic switcher with the same BP duty cycle, including radio and weight-download energy	Hybrid uses less edge energy than the periodic switcher at equal mAP, across 5 or more seeds	Report the regime where adaptive switching helps and where it does not

Proposed order: Gate 2 microbenchmark first (it is the largest risk), then Gate 1 in parallel on Azure, then Gate 3.

Deliverables for the meeting: the three gates above with agreed thresholds, a one-page results table per gate, and a plot of energy saving against disturbance fraction.

Revised system architecture

A runtime hysteresis state machine picks the mode, and J is an offline metric used only to compare operating points. This revises the proposal you shared: the payload size, energy constants and supervision source are now measured or stated, not assumed.

Tier 1: runtime policy. The controller works on a normalized free energy, F_n(t) = F(t) / F_ref(t), where F_ref is an EWMA of F taken only during stable Mode 0 windows. This replaces fixed thresholds tuned on MNIST.

```
\mathrm{Mode}(t)=\begin{cases}1 & F_n(t)>\theta_{\mathrm{up}}\\[2pt] 0 &
F_n(t)<\theta_{\mathrm{low}}\\[2pt] \mathrm{Mode}(t-1) &
\text{otherwise}\end{cases}
\qquad \theta_{\mathrm{up}}=\tau_{\mathrm{upper}}+H,\quad
\theta_{\mathrm{low}}=\tau_{\mathrm{lower}}-H
```

- **Two effective thresholds:** only theta_up and theta_low matter, so H is not a separate dimension.
- **Mode 0 (edge):** PC adapts the bottleneck features locally. Network egress is 0 bytes.
- **Mode 1 (offload):** the edge sends quantized bottleneck features by HTTP POST to the Azure VM (Standard_D4as_v5). The cloud trains the adapted module and returns only the weight delta. Report both payload sizes as measured values. For reference, a 256x20x20 fp32 feature map is about 400 KB, or about 100 KB in int8.
- **Monitoring cost:** the edge keeps computing F in Mode 1 so it can switch back, and that cost is counted in E.
- **Anomaly guard:** use the spatial F map. A global rise (lighting, blur) can trigger Mode 1, but a localized spike (an object entering) must not trigger adaptation, so the model does not learn the intruder away.

Tier 2: offline objective.

```
J=\alpha\,\mathrm{clip}\!\left(\tfrac{E}
{E_{\mathrm{ref}}}\right)+\beta\,\mathrm{clip}\!\left(\tfrac{L}
{L_{\mathrm{ref}}}\right)+\gamma\,(1-
A)+\delta\,\mathrm{clip}\!\left(\tfrac{F}{F_{\mathrm{ref}}}\right),\qquad
\alpha+\beta+\gamma+\delta=1
```

- **Reference values:** define E_ref, L_ref and F_ref explicitly (for example the Always-BP energy, the real-time latency budget, and the largest F seen in training streams) and clip each term to [0, 1].
- **Weights:** sweep them over the simplex and plot the Pareto front. The weights (0.45, 0.15, 0.20, 0.20) are one example point, not the analysis. Also report the "minimize energy subject to mAP at or above a target" version.
- **Tuning:** tune thresholds on one set of streams and report results on a separate set.

Supervision.

- **Mode 0:** self-supervised. PC minimizes feature prediction error with no labels.
- **Mode 1:** the main source is pseudo-labels from a larger cloud teacher (for example YOLOv8x), using the YOLO detection loss (box, class and DFL terms). Pseudo-labels from tracked detections in the prior stable window are an ablation, because during a disturbance objects have moved and self-training reinforces its own errors.
- **Evaluation:** mAP is computed against ground-truth labels, never against pseudo-labels.

Energy model. Each term is a measured component, not one constant power.

```
E^{(0)}_{\mathrm{frame}}=E_{\mathrm{backbone}}+E_{\mathrm{PC}}+E_{\mathrm{monitor}},\qquad
E^{(1)}_{\mathrm{frame}}=E_{\mathrm{backbone}}+E_{\mathrm{monitor}}+E_{\mathrm{tx}}+E_{\mathrm{rx}}+E_{\mathrm{wait}}
```

Each component is E_x = P_x * T_x, with P_x measured on the Pi 5 with the USB-C power meter and idle power subtracted. Report the model's error against total measured joules per run (for example MAPE).

Quantity	How to obtain it	Replaces
P_backbone, P_PC, P_monitor	Pi 5 power meter, 5 or more repeats, idle subtracted	the flat 5.0 W
P_tx, P_rx and Wi-Fi tail energy	Pi 5 power meter during real transfers over Wi-Fi and Ethernet	the flat 1.2 W
Uplink bytes	measured from the actual quantized feature payload	the assumed ~1 KB
Downlink bytes	size of the returned weight delta	not modeled before
E_wait	edge power while waiting for the cloud round trip, from `tc netem` runs	not modeled before

Fixes to the blueprint and report

- **Blueprint PDF equations:** they render as raw LaTeX (for example `rac{1}{N}` and `\mathbf{\checkmark}`). Fix this before it goes to your professor.

- **Titles and timeline:** the blueprint says "Edge IoT Surveillance Systems", the report says "Embedded Edge Vision", and your own wording differs again. The timeline is 6 months in one document and 10 weeks in the other.
- **References (not verified by me):** the Salvatori entry has an arXiv ID from 2021 labeled 2024, and HIT-UAV is a drone thermal dataset unrelated to this work. Cite a source for the "80-90% of energy is memory access" claim (Horowitz 2014 or Sze et al. 2017 are the usual ones).
- **Pi power code:** to my knowledge `vcgencmd pmic_read_adc` reports EXT5V as voltage only, with currents per internal rail, so the `ext5v_i` regex likely matches nothing and the function silently returns 0.0. Run the command once and read the output. Sample power during the workload instead of once afterward, and cross-check against the USB meter.
- **Camera loop:** `cv2.getTickCount() & 0xFF == ord('q')` should be `cv2.waitKey(1) & 0xFF == ord('q')`.